Gemini API — Hackathon Setup Guide

For teams building on **self-hosted n8n**, **Gumloop**, **Claude Code**, and **Replit**.

Find your tool below and follow that section. You only need yours.

---

1. Your API key

Three keys are available. **Check with the organizers which one your team uses.** Free-tier rate limits are counted per key, so if all teams pile onto one key you will throttle each other and spend the event staring at `429` errors.

Key
`AQ.Ab8RN6Jm-I-AD2_E9QqM2QIkL-wfV37Ocsw66Tu1Y8pLOavnEw`
`AQ.Ab8RN6I96FSxyxpQb9Z4-nqREaURcP-cIOugW2qTf_mKKtFjsw`
`AQ.Ab8RN6IU02GOmSpUw5q2mIPC9mOR4Q0wjD8AYlTJnc44U55UWQ`

These are shared event credentials. Handling rules are in section 9 — please read them, they're short.

---

2. The three values you'll be asked for

Every tool below wants some combination of these. Nothing else.

Field	Value
API host / base URL	`https://generativelanguage.googleapis.com`
API key	key from section 1
Model ID	`gemini-3.5-flash` to start (full list in section 7)

If a tool asks for a header name for auth, it's `x-goog-api-key`.

---

3. Self-hosted n8n

n8n has a native Gemini node, so there's nothing to build by hand.

Create the credential (once per instance)

1. Go to **Credentials** → **Add credential**.
2. Search for **Google Gemini(PaLM) API** and select it.
3. Fill in two fields:
   - **Host** — `https://generativelanguage.googleapis.com`
   - **API Key** — your assigned key
4. Save. n8n encrypts credentials at rest, so the key isn't sitting in plaintext inside your workflow.

Self-hosted note: the *Use Gateway credits* option you'll see in some tutorials is n8n Cloud only. On a self-hosted instance you must supply your own key — which is what these keys are for.

Also: the Gemini nodes don't support custom hosts or proxies. The host field has to be exactly the URL above. If your instance sits behind an egress firewall, allow `generativelanguage.googleapis.com` outbound, or every node fails with a connection error and no useful message.

Use it in a workflow

- **Google Gemini** node → operation **Message a Model** is the simplest text call.
- The same node also covers **Analyze Image**, **Analyze Document**, **Analyze Audio**, **Transcribe a Recording**, and uploading to a File Search store for RAG.
- For AI Agent or chain workflows, use the **Google Gemini Chat Model** sub-node and connect it to the agent's model input.
- For embeddings plus a vector store, connect an **Embeddings Google Gemini** sub-node to the vector store's embeddings input.

If the node lacks what you need

Use the **HTTP Request** node and reuse the same credential rather than pasting the key again: set **Authentication** → **Predefined Credential Type**, pick your Google Gemini credential, then point the node at the endpoint you need. Endpoints are in section 8.

Before you demo

Export your workflow and confirm the key isn't embedded anywhere in the exported JSON. It shouldn't be, but check — sharing a workflow export is a common way keys escape.

---

4. Gumloop

Read this first — plan requirement

Bring-your-own-key on Gumloop requires a **paid plan (Starter/Solo or above)**. On a free plan you can't add a Gemini key; AI nodes run on Gumloop's own credentials and consume credits.

If your team is on a free Gumloop plan, these keys are of no use to you there. Either get on a paid plan before the event, budget for the included credits, or plan to make your AI calls from a different tool. Sort this out now, not on hackathon morning.

Add the key

1. Go to **Connectors** (or **Settings** → **Profile** → **Connectors**).
2. Find **Google Gemini** in the available integrations.
3. Add your assigned key and save.

Scope matters:

- **Personal** — only your own runs use it.
- **Team / workspace** — shared, so the whole team benefits without each person adding it. Usually the right choice for a hackathon team.

- **Organization → API keys** — admin level, overrides personal and team keys for everyone. Only relevant if an admin is setting this up centrally.

What this changes

With a Gemini key connected, AI model calls cost **0 credits** in both workflow AI nodes and agents, including image generation and voice transcription. Agent chats on BYOK carry a higher orchestration fee than non-BYOK runs, but the total is still a large net saving because the model spend leaves your credit balance entirely.

Using it

The key is picked up automatically by AI nodes — **Ask AI**, **Analyze Image**, **Generate Report** and friends — whenever you select a Gemini model in the node's model picker.

Don't confuse two different keys. The *Gumloop API Key* (also under Connectors, used to trigger your flows via webhook, CLI, or SDK) is a completely separate thing from the Gemini provider key. If a guide tells you to generate an API key, check which one it means.

---

5. Claude Code

Important framing: Claude Code runs on Anthropic's models. You are not putting a Gemini key *into* Claude Code to power it. The Gemini key is for the **app Claude Code builds for you** — Claude Code is your coding agent, Gemini is an API your app calls.

Setup

1. In your project root, put the key in a `.env` file as `GEMINI_API_KEY`.
2. **Add `.env` to `.gitignore` before your first commit.** Do this first, not later.
3. Create a `.env.example` alongside it with the variable name and no value, so teammates know what to fill in.
4. Write the convention into your `CLAUDE.md`: read the Gemini key from the environment, never hardcode it, never log it, and route all Gemini calls through a single server-side module rather than scattering them through the codebase. Claude Code will follow this consistently once it's written down.

Environment variables can also go under the `env` key in a settings file. Put anything secret in `.claude/settings.local.json` — that's the personal, non-committed layer, whereas `.claude/settings.json` is meant to be shared via git. Add `.claude/settings.local.json` to `.gitignore` too.

Two things that bite people

Claude Code can read your `.env`. It's a normal file and normal file-reading tools apply, so a key stored there can end up in the conversation transcript if the agent reads it. If that matters to you, add a deny rule for `.env` under `permissions.deny` in `.claude/settings.json`. A `.claudeignore` file is *not* a supported mechanism for this — don't rely on it.

Agents are enthusiastic committers. Review what goes into a commit before pushing, especially if you've been letting Claude Code run git commands. Ask it to show you the diff first.

One tip

If you want Claude Code to write the Gemini integration, point it at the docs links in section 11 rather than letting it work from memory. Google's model IDs move fast and it'll otherwise reach for a deprecated one and hand you a `404`.

---

6. Replit

Add the key

1. Open your Replit App.
2. In the left tool dock, open **Secrets** (padlock icon). If you don't see it: **All tools** → **Secrets**, or search "Secrets".
3. Click **New Secret**.
4. **Key:** `GEMINI_API_KEY` — **Value:** your assigned key.
5. Click **Add Secret**.

Replit encrypts secrets with AES-256 at rest and TLS in transit, and exposes them to your app as environment variables at runtime. They aren't visible in your source files, and visitors to a public App's cover page can't see them.

You can add secrets at **App level** (this project only) or **Account level** (available across all your Apps). App level is right for a hackathon project.

Three Replit gotchas

Workspace secrets do not carry over to deployments. This is the single most common cause of "it worked in the editor and broke during the demo". When you deploy, add the secret again in the **Deployments** pane under its own Secrets section.

Secrets aren't hidden from collaborators. Non-owner members can't *view* secret values in the UI, but they can print environment variables from the shell or from code. Anyone with edit access effectively has the key.

Forks don't carry secrets. If someone remixes your App they get the code and no values. Good for safety, but expect to re-add the secret if you fork your own project.

Working with Replit Agent

Replit Agent has its own managed AI credentials for AI features it builds, billed to your Replit account. That's separate from this. To make it use *your* Gemini key, say so explicitly in your prompt: read `GEMINI_API_KEY` from Replit Secrets, call the Gemini API from a backend route only, and never expose the key to the frontend.

Never let the key reach client-side code. If Agent wires a Gemini call directly into your React or frontend JS, the key ships to the browser and anyone can read it from devtools. Ask for a backend route that the frontend calls instead — and check the generated code, because Agent gets this wrong sometimes.

---

7. Model IDs

Use case	Model ID
Good default	`gemini-3.5-flash`
Newest, strongest Flash	`gemini-3.8-flash`
Cheapest / fastest	`gemini-3.5-flash-lite`
Image generation + editing	`gemini-3.1-flash-image`

Embeddings (RAG, search)	`gemini-embedding-001`
Heaviest reasoning	`gemini-3.1-pro-preview`

Start on `gemini-3.5-flash`. It's stable and has the widest example coverage online, which matters a lot when you're debugging under time pressure. Move up to `gemini-3.8-flash` if you need better coding or agentic performance.

Two cautions:

- `gemini-3.1-pro-preview` is preview status, so rate limits are tighter. Not a good pick for a room full of teams hitting it at once.
- Avoid `latest` aliases like `gemini-flash-latest`. They get hot-swapped when Google ships a new version — exactly the surprise you don't want mid-demo.

If a tool's model dropdown doesn't list what you want, most of them let you type a model name manually. Copy the ID exactly; a typo returns `404`, not a helpful message.

---

8. Endpoint reference

Only needed if you're using an HTTP Request node, a custom connector, or hand-rolling the call. Base URL is `https://generativelanguage.googleapis.com`.

Purpose	Method + path
Text / multimodal generation	`POST /v1beta/models/{model}:generateContent`

Streaming (SSE)	`POST /v1beta/models/{model}:streamGenerateContent`
Embeddings	`POST /v1beta/models/{model}:embedContent`
Count tokens	`POST /v1beta/models/{model}:countTokens`
List available models	`GET /v1beta/models`
Batch jobs	`POST /v1beta/models/{model}:batchGenerateContent`
File upload	`POST /v1beta/files`

Auth is the `x-goog-api-key` header. The older `?key=` query parameter still works but avoid it — it leaks into server logs and proxy traces.

Google now recommends its newer **Interactions API** (`POST /v1beta/interactions`) as the standard primitive for agentic work with server-side conversation state. For a hackathon, stick with `generateContent`: stateless, easier to debug, far more community examples. It remains fully supported.

There's also an OpenAI-compatible layer at `/v1beta/openai/`. If a tool only offers an "OpenAI-compatible endpoint" field, use that as the base URL, your Gemini key as the API key, and the Gemini model ID in the model field.

---

9. Key handling rules

Short list, all of it matters:

- **Never hardcode the key** in source, workflow exports, or a prompt. Use the secret store your tool gives you — n8n credentials, Gumloop connectors, Replit Secrets, `.env`.
- **Never put it in frontend code.** Anything in the browser is public. Backend routes only.
- **Never commit it.** `.gitignore` your `.env` before the first commit. GitHub and Google both scan public repos for exposed credentials and auto-revoke what they find, usually at the worst possible moment.
- **Don't paste it into a public channel, a screenshot, or a screen share.** Close your Secrets pane before demoing.
- **Don't switch to another team's key** when yours rate-limits. Tell the organizers instead.

If a key stops working mid-event, assume it leaked and was revoked. Fastest recovery is for someone on the team to generate a fresh personal key at https://aistudio.google.com/apikey — about four clicks, no credit card.

---

10. Common errors

Code	Meaning	Fix
`401` / `403`	Bad, revoked, or misplaced key	Re-paste the key, check for trailing spaces, confirm the credential is actually selected on the node
`404`	Unknown model ID	Copy the ID exactly from section 7
`429 RESOURCE_EXHAUSTED`	Rate limited	Wait and retry, switch to `flash-lite`, confirm nobody else is on your key

`400 INVALID_ARGUMENT`	Malformed request	Usually an empty prompt field or an unfilled expression in a no-code node
`500` / `503`	Transient Google-side issue	Retry, then check the status page
Connection timeout	Egress blocked (self-hosted n8n especially)	Allow `generativelanguage.googleapis.com` outbound

---

11. Links

- API reference — https://ai.google.dev/api
- Model list — https://ai.google.dev/gemini-api/docs/models
- Rate limits — https://ai.google.dev/gemini-api/docs/rate-limits
- Get your own key — https://aistudio.google.com/apikey
- Service status — https://aistudio.google.com/status
- n8n Gemini node — https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-langchain.googlegemini
- n8n Gemini credential — https://docs.n8n.io/integrations/builtin/credentials/googleai
- Gumloop AI models & BYOK — https://docs.gumloop.com/core-concepts/ai_models
- Replit Secrets — https://docs.replit.com/replit-workspace/workspace-features/secrets
- Claude Code docs — https://docs.claude.com/en/docs/claude-code/overview